

DeSFAM: An Adaptive eBPF and AI-Driven Framework for Securing Cloud Containers in Real Time

SEHAR ZEHRA¹, HASSAN JAMIL SYED², FAHAD SAMAD³, AND UMMAY FASEEHA¹, (Member, IEEE)

Abstract—Containerized applications offer lightweight and scalable deployment but remain exposed to security risks due to a shared kernel. We present DeSFAM (Dynamic eBPF-driven Syscall Filtering and Anomaly Mitigation), a real-time security framework that enforces least-privilege syscall usage and detects behavioral anomalies. DeSFAM integrates: 1) hybrid syscall profiling through static analysis and dynamic eBPF tracing; 2) SyscallAD (System call Anomaly Detection), a low-latency anomaly detector combining Variational Autoencoder (VAE) and Isolation Forest (iForest); 3) contextual risk scoring based on MITRE ATT&CK mappings and CVE correlations; and 4) adaptive syscall enforcement using eBPF maps and LSM hooks. Evaluations using the DongTing dataset and real-world CVE attack scenarios show DeSFAM achieves 94% precision, 90% recall, sub-millisecond enforcement latency, and less than 1% performance overhead. DeSFAM effectively blocks privilege escalation, container escape attempts, and syscall injection attacks in modern container environments.

Index Terms—Anomaly detection, cloud containers, container security, eBPF monitoring, real-time threat detection, syscall filtering, seccomp hardening.

I. INTRODUCTION

Containerization has become a cornerstone of cloud-native computing due to its efficiency and scalability. Unlike traditional virtual machines, containers share the host OS kernel, leading to reduced overhead but also introducing critical security risks through system calls (syscalls)—the interface between user space and the kernel [1], [2].

Syscalls expose a broad attack surface, making them attractive targets for privilege escalation, container escape, and other exploits [3]. Existing solutions like seccomp profiles offer only limited protection. Static profiles are

hard to maintain and may block legitimate operations or fail to detect sophisticated threats.

To overcome these limitations, we propose DeSFAM, Dynamic eBPF-Driven Syscall Filtering and Anomaly Mitigation. DeSFAM provides real-time syscall profiling, deep anomaly detection, and adaptive policy enforcement. It integrates:

- (i) Hybrid Syscall access listing using static and eBPF-based dynamic analysis;
- (ii) SyscallAD, a detection engine combining Variational Autoencoder (VAE) and Isolation Forest (iForest) for identifying anomalous syscall sequences; and
- (iii) Adaptive Policy Management that uses MITRE Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK) framework mappings, Common Vulnerabilities and Exposures (CVE) correlations, and eBPF LSM hooks for context-aware syscall control.

DeSFAM enforces fine-grained syscall control using eBPF maps and LSM hooks, dynamically blocking high-risk syscalls (e.g., `execve`, `ptrace`) and updating policies at runtime without requiring container restarts. It combines a hybrid syscall profiling pipeline with SyscallAD, a low-latency anomaly detector based on Variational Autoencoder (VAE) and Isolation Forest. To prioritize threats, DeSFAM integrates contextual risk scoring by correlating syscall behaviors with MITRE ATT&CK tactics and known CVEs. Evaluations using the DongTing dataset and real-world CVE scenarios demonstrate 94% precision, 90% recall, sub-millisecond enforcement latency, and under 1% performance overhead.

Our key contributions are:

- A hybrid profiling pipeline for syscall allowlist generation.
- SyscallAD, a low-latency VAE + iForest anomaly detector.
- Contextual risk scoring using MITRE ATT&CK and CVE correlations.
- Fine-grained syscall enforcement using eBPF maps and LSM hooks.
- A runtime feedback loop for policy auto-hardening based on violations.

The paper is organized as follows: Section II covers syscall filtering and eBPF fundamentals. Section III dis-

¹Department of Computer Science, FAST National University of Computer and Emerging Sciences, Karachi 75030, Pakistan

²Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur 57000, Malaysia

³Department of Cyber Security, FAST National University of Computer and Emerging Sciences, Karachi 75030, Pakistan
Corresponding author: Hassan Jamil Syed (has-san.jamil@apu.edu.my)

This work was supported by the Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur, Malaysia.

Original publication: *IEEE Access*, vol. 13, 2025, DOI: 10.1109/ACCESS.2025.3592192

cusses prior work. Section IV describes DeSFAM’s architecture. Section V presents evaluation results, followed by discussion in Section VI and conclusion in Section VII.

II. BACKGROUND

Containerization has fundamentally transformed cloud-native software deployment by offering lightweight, scalable, and portable execution environments. Unlike virtual machines (VMs), which encapsulate a full operating system per instance, containers share the host’s OS kernel while providing isolated user spaces via namespaces and control groups (cgroups). This architecture reduces overhead and accelerates deployment, making containers ideal for microservices, CI/CD pipelines, and elastic workloads. However, this shared kernel model introduces unique security risks—particularly at the syscall interface, which governs access to core kernel functionalities.

A. System Call Security in Shared-Kernel Environments

Syscalls act as the primary interface between user-space applications and kernel resources. In containerized systems, they are critical to process execution, file operations, memory management, and network communication. Unfortunately, their broad capabilities also make them a prime target for attackers. The kernel exposes hundreds of syscalls, many of which are unnecessary for typical containerized workloads, thereby expanding the attack surface [4].

Attackers often exploit misused or vulnerable syscalls to execute privilege escalation, escape containers, or tamper with the host environment. Such threats are exacerbated in multi-tenant cloud deployments, where multiple containers share the same host kernel and thus the same syscall namespace. Without rigorous syscall control, a compromise in one container can lead to lateral movement or host takeover.

B. Types of Syscall-Based Threats

System calls expose core kernel functions, making them prime targets in containerized environments. Attackers exploit them to escalate privileges, escape isolation, or abuse over-permissive profiles.

Privilege Escalation: Attackers may exploit kernel vulnerabilities, misconfigured Linux capabilities, or incomplete seccomp profiles to gain elevated privileges from within a container. Examples include the use of `setuid()`, `clone()`, or `execve()` in specific sequences to hijack thread execution or bypass namespace restrictions.

Container Escape: Exploits like CVE-2022-0847 [5] (Dirty Pipe) or CVE-2019-5736 [6] (Docker runc) allow malicious processes to bypass container boundaries and interact with the host. These often involve crafting malicious syscall sequences that tamper with read-only memory or escape namespace isolation.

Attack Surface Abuse: Many containers have overly broad access to syscalls due to generic or default seccomp

profiles. This over-permissiveness can be abused by sophisticated adversaries to chain together syscalls into stealthy, multi-stage attacks.

C. Limitations of Traditional Mitigation Approaches

Security mechanisms such as AppArmor, SELinux, and Seccomp provide syscall filtering and policy enforcement. However, they exhibit several limitations in modern cloud-native environments:

- **Static Policy Inflexibility:** Static profiles do not adapt to new behaviors or threat vectors, often resulting in false positives (over-blocking) or false negatives (missed attacks).
- **Lack of Sequence Awareness:** Rule-based systems operate on individual syscall invocations without considering temporal dependencies, making them blind to multi-step attacks.
- **Scalability Bottlenecks:** As container density increases, static filters become harder to maintain and enforce efficiently at scale.
- **Manual Configuration Overhead:** Policies must be manually written, tested, and maintained for each container workload—an impractical burden in dynamic environments.

These shortcomings highlight the need for dynamic, intelligent, and context-aware syscall security solutions that scale with modern infrastructure.

D. The Promise of eBPF for Syscall Monitoring

eBPF is a kernel-native technology that allows safe, programmable instrumentation of kernel events, including syscalls [7]. It enables dynamic attachment of monitoring and filtering logic directly at syscall entry points (e.g., `sys_enter_*`) with minimal overhead. Key benefits of eBPF in the syscall security domain include:

- **Real-Time Enforcement:** Syscalls can be blocked or audited instantly without context switches or restarts.
- **Context-Aware Filtering:** Policies can account for namespaces, cgroup IDs, arguments, or syscall patterns.
- **High Scalability:** Efficient in-kernel maps and tracepoints allow enforcement over thousands of containers concurrently.

eBPF’s programmable nature makes it an ideal foundation for implementing fine-grained, real-time syscall security tailored to container workloads.

E. Sequence-Based Anomaly Detection for Syscalls

Machine learning (ML) models, especially those analyzing syscall sequences, provide a powerful complement to eBPF monitoring. Rather than relying solely on predefined rules, sequence-based models can learn patterns of normal behavior and flag anomalous syscall windows that deviate from expected workflows.

Variational Autoencoders (VAE) [8] and Isolation Forests [9] (iForest) are particularly effective for modeling syscall sequences:

- VAEs reconstruct normal behavior from compressed latent space representations. High reconstruction error indicates anomalies.
- Isolation Forests identify statistical outliers based on how easily a sequence can be separated from normal patterns.
- Ensembles of both improve robustness by combining reconstruction-based and tree-based anomaly scoring.

To support training and validation, public datasets such as the DongTing dataset [10] offer syscall sequences labeled as normal or malicious. DeSFAM uses DongTing exclusively, leveraging its 18,966 labeled sequences to train and calibrate its anomaly detection models.

F. Toward Adaptive, Scalable Syscall Security

The dynamic nature of container orchestration demands syscall filtering mechanisms that adapt to workload behaviors, incorporate threat intelligence, and scale with minimal human intervention.

DeSFAM addresses these needs by combining:

- Hybrid static-dynamic syscall profiling.
- Ensemble anomaly detection combining deep learning (VAE) and tree-based models (iForest).
- Risk-based policy enforcement using eBPF LSM hooks.
- CVE-matching and MITRE ATT&CK-based risk scoring.
- Auto-hardening mechanisms to refine seccomp profiles at runtime.

This integrated design enables real-time detection, contextual blocking, and long-term syscall surface reduction, setting the stage for resilient and scalable container security in shared-kernel environments.

III. RELATED WORK

Research on host-based intrusion detection using system calls spans three decades and can be grouped into four directions: sequence-based statistical methods, deep-learning anomaly detectors, container-oriented approaches, and the DeSFAM framework that this work reproduces and extends. Table I summarises the seven most closely related methods, comparing their technique, evaluation dataset, best reported metric, and key limitation.

A. Classic Sequence-Based Methods

Forrest et al. [11] introduced STIDE, the foundational approach in host-based IDS: normal behaviour is modelled as a set of short n -gram syscall sequences, and any unseen sequence triggers an alarm. Follow-on work by Hofmeyr et al. [18] and Warrender et al. [19] extended STIDE and compared it against Hidden Markov Models (HMMs), finding that HMMs yield the lowest false-positive rate

but are too slow for real-time deployment. Wagner and Soto [20] later demonstrated that sequence-only models are fundamentally vulnerable to mimicry attacks, where an attacker interleaves benign system calls to make malicious traces appear normal.

Wang et al. [12] proposed Anagram, which replaces the deterministic lookup table with randomised n -gram Bloom filters, providing statistical resistance to mimicry while reducing memory overhead. Nevertheless, both STIDE and Anagram share a fundamental weakness: they model call *order* but ignore call *arguments*, frequency distribution, and temporal context—features that a VAE can capture through latent representations.

B. Deep Learning for Syscall Anomaly Detection

Creech and Hu [13] replaced sequence matching with semantic features derived from context-free grammar analysis, fed into an Extreme Learning Machine (ELM). Their method achieves state-of-the-art results on ADFA-LD at the time, but requires days of training per host and does not adapt to concept drift—making it impractical for dynamic container workloads.

Zong et al. [14] proposed DAGMM, a jointly trained deep autoencoder and Gaussian Mixture Model that achieves up to 14% F1 improvement over prior SOTA on KDD Cup 99. Although DAGMM demonstrates the value of unsupervised deep anomaly detection, it is evaluated only on tabular network-flow data and has not been applied to raw syscall traces.

Haider et al. [15] conducted the most comprehensive deep-learning comparison on ADFA-LD, testing LSTM (200/400-cell), GRU, CNN+RNN, and WaveNet. WaveNet achieves the highest accuracy on ADFA-LD; however, all models are *supervised*—they require labelled attack traces at training time, which is infeasible in production deployments where attack sequences are unavailable or unlabelled.

C. Container-Oriented and Unsupervised Approaches

Abed et al. [21] applied Bag-of-System-Calls with k -NN and k -means to Docker container traces, showing that even order-agnostic frequency counts outperform some sequence methods in container contexts. Castanhel et al. [22] demonstrated that pre-filtering to 1% of syscall traces via relevance scoring maintains >90% F1, offering a lightweight deployment path. El Khairi et al. [16] (CHIDS) combine heterogeneous syscall property vectors with an unsupervised autoencoder and achieve high AUC across multiple attack scenarios, but evaluate on a private single-cluster dataset with no public benchmark. Bsoul et al. [23] construct directed syscall graphs and apply a Random Forest and Isolation Forest ensemble, yielding high detection rates across seven attack types; however, the Random Forest component requires labelled attack data.

D. DeSFAM and the Gap Addressed by This Work

Mansour et al. [17] introduced DeSFAM, an end-to-end eBPF framework that combines hybrid syscall profiling,

TABLE I
COMPARISON OF REPRESENTATIVE SYSCALL ANOMALY DETECTION METHODS

Method	Year	Technique	Dataset	Key Limitation
STIDE [11]	1996	n -gram sequence lookup table	UNM traces	Evaded by mimicry attacks; no argument context
Anagram [12]	2006	Randomised n -gram Bloom filters	UNM custom	High-order n -grams costly; no semantic context
Creech & Hu [13]	2014	Grammar phrases + Extreme Learning Machine	ADFA-LD	Training takes days per host; not drift-robust
DAGMM [14]	2018	Deep AE + Gaussian mixture estimation	KDD Cup 99	Evaluated on tabular data only; not on syscall traces
Haider et al. [15]	2021	LSTM / WaveNet on syscall sequences	ADFA-LD, PLAID	Supervised: requires labelled attack data for training
CHIDS [16]	2022	Unsupervised AE on syscall property vectors	Custom container	Private dataset; per-container profiling required
DeSFAM [17]	2025	VAE + Isolation Forest ensemble (SyscallAD)	DongTing	Evaluated on single dataset; cross-dataset gap unstudied

SyscallAD (a VAE and Isolation Forest ensemble), CVE-aware filtering, and LSM-based enforcement. SyscallAD is trained exclusively on normal traffic and evaluated on DongTing [10], achieving Precision 94% and Recall 90% with sub-millisecond enforcement latency.

Despite these results, no published work has evaluated SyscallAD on any dataset outside DongTing. As Table I shows, all existing HIDS papers either use private container datasets or a single public benchmark. This leaves two open questions: (1) does SyscallAD’s accuracy hold on a structurally different benchmark such as ADFA-LD, which contains real-world attack categories (brute-force, lateral movement, web shell) rather than kernel bug triggers? (2) which component—VAE or Isolation Forest—is responsible for performance degradation when the attack distribution changes?

This work addresses both questions by reproducing SyscallAD on DongTing to verify its published results, and extending the evaluation to ADFA-LD to characterise cross-dataset generalisation. Our experiments reveal that the Isolation Forest component collapses on ADFA-LD ($AUC = 0.33$), while the VAE component retains discriminative ability ($AUC = 0.67$), identifying component-specific failure modes as a finding not reported in any prior work.

IV. DeSFAM: DYNAMIC EBPF-DRIVEN SYSCALL FILTERING AND ANOMALY MITIGATION

This section presents the design and implementation of the proposed DeSFAM framework, detailing its architecture, operational phases, and enforcement strategy.

A. Overview

DeSFAM (Dynamic eBPF-Driven Syscall Filtering and Anomaly Mitigation) is a lightweight, real-time framework designed to reduce the kernel attack surface and detect anomalous syscall behavior in containerized environments. It enforces least-privilege syscall usage to mitigate threats such as privilege escalation, container escape, and syscall

injection. Fig. 1 illustrates the architectural workflow of the framework.

DeSFAM operates in three integrated phases:

- **Hybrid Syscall Access Listing and Seccomp Profiling:** Combines static call graph extraction and dynamic eBPF tracing to construct a refined syscall access list, pruning unnecessary and CVE-associated syscalls.
- **Real-Time Monitoring and Anomaly Detection (SyscallAD):** Traces runtime syscall sequences and analyzes them using a VAE + Isolation Forest ensemble to detect sequence-level anomalies, including malicious use of permitted syscalls.
- **Adaptive Enforcement and Policy Management:** Enforces blocking, rate-limiting, or logging of high-risk syscalls using eBPF maps and LSM hooks, guided by risk scores from SyscallAD, without disrupting container execution.

By integrating sequence-aware profiling, anomaly detection, and real-time enforcement, DeSFAM offers scalable, adaptive protection suited for modern cloud-native container deployments.

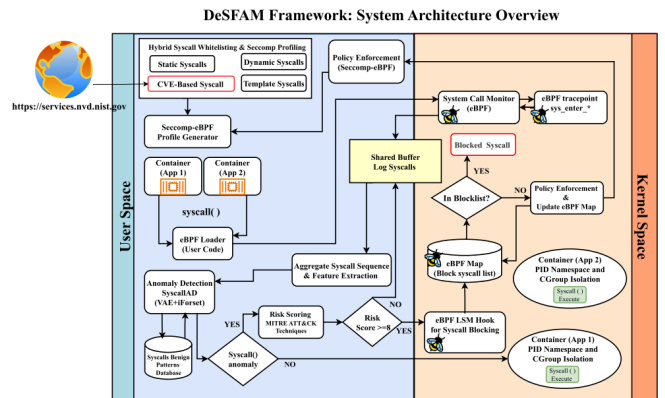


Fig. 1. DeSFAM architecture showing the interaction between user-space syscall profiling, anomaly detection, and kernel-space enforcement using eBPF and seccomp for adaptive, real-time syscall control.

B. Design Considerations

The design of DeSFAM is guided by the need to enforce syscall least privilege while addressing the limitations of static-only filtering, high-overhead provenance tracking, and retraining-dependent anomaly detectors. Traditional seccomp profiles are either too permissive or brittle, failing to adapt to evolving runtime behavior. DeSFAM overcomes this by combining static and dynamic analysis, enabling accurate and context-aware syscall profiling.

To ensure real-time enforcement without impacting container availability, DeSFAM leverages eBPF probes and LSM hooks, which operate entirely in kernel space and support dynamic policy updates with minimal latency. This design choice avoids the performance cost of continuous provenance graph construction and eliminates the need for kernel modification or container restarts.

For anomaly detection, DeSFAM adopts a hybrid model (VAE + Isolation Forest) that captures behavioral patterns in syscall sequences while avoiding frequent retraining, supporting scalable and robust detection of zero-day attacks. Unlike hardware-bound solutions such as TEEs, DeSFAM remains fully software-based and portable, ensuring compatibility across a wide range of cloud-native deployments.

C. Phase 1: Hybrid Syscall Access Listing and Seccomp Profiling

Container security hinges on precise syscall control, as containers share the host kernel and pose risks of unauthorized kernel access. Phase 1 of DeSFAM mitigates this by constructing a container-specific syscall access list through a hybrid approach combining static analysis, dynamic profiling, and CVE-based threat intelligence to generate a minimal and accurate seccomp profile.

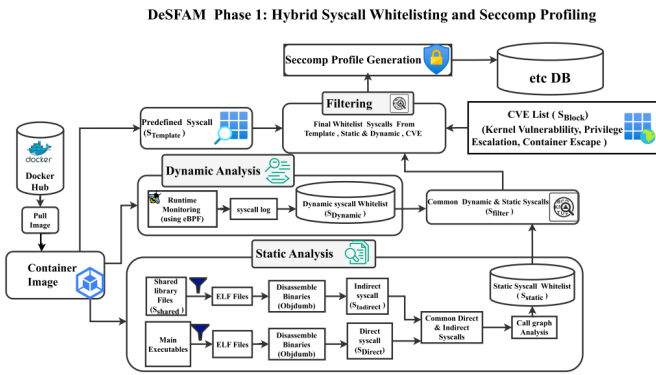


Fig. 2. DeSFAM Phase 1: Hybrid syscall access listing and seccomp profiling using static analysis, eBPF monitoring, and CVE-informed filtering to generate secure container-specific syscall policies.

1) *Static Analysis*: Static analysis begins by examining container images, including Dockerfiles, binaries, and linked libraries, without executing them. Using tools like `objdump` and `readelf`, ELF binaries are disassembled to extract low-level syscall instructions and libc wrapper functions.

To enhance coverage, a Static Call Graph (SCG) is constructed using NetworkX to trace function-to-syscall mappings across the codebase. The resulting static syscall set, S_{static} , captures all identifiable syscalls reachable from the code's control flow.

- **Input:** Container image (Dockerfile, ELF binaries)
- **Tools:** `objdump`, `readelf`, `networkx`
- **Process:** Disassemble ELF binaries, extract syscall paths via SCG traversal
- **Output:** Static syscall list (S_{static})

However, static analysis cannot detect syscalls invoked by runtime-resolved libraries or JIT-compiled code. Thus, while S_{static} offers a foundational syscall set, it must be refined through dynamic analysis in subsequent phases.

2) Dynamic Profiling: Capturing Runtime Behavior:

To address the limitations of static analysis, DeSFAM incorporates dynamic profiling by monitoring live syscall activity during container execution under standardized representative workloads. This phase captures syscalls triggered by runtime conditions including dynamically loaded libraries, environment-specific logic, and dependency interactions, while ensuring comprehensive coverage of typical operational behaviors.

Containers execute under role-specific workloads designed using industry-standard benchmarks that reflect real-world operational profiles:

- **Web servers (Nginx):** `wrk` with 500–10,000 concurrent connections; ApacheBench simulating GET/POST requests at 1K–10K req/sec.
- **Databases (MySQL/Redis/MongoDB):** SysBench OLTP workloads (100–500 threads, 1M+ transactions); `redis-benchmark` at 1M ops/sec.
- **Application runtimes (Node.js/Python):** Locust load tests simulating 50–500 concurrent user flows; real-world dataset processing (GB-scale JSON/CSV parsing).

These workloads cover 90th-percentile production traffic patterns, container startup/shutdown sequences, dependency interactions, and failure scenarios.

During execution, eBPF probes are attached to syscall entry points (`sys_enter_*`) to trace syscall events. The collected metadata includes syscall IDs, process IDs, timestamps, and namespace identifiers.

- **Input:** Container instances under standardized workloads
- **Tools:** eBPF via `bcc/bpftrace` with kernel-space tracing
- **Process:** Deploy containers → attach eBPF probes → log metadata to ring buffers → aggregate syscalls across 3 workload cycles
- **Output:** Dynamic syscall list ($S_{dynamic}$)

3) Threat Intelligence: CVE-Based Syscall Filtering:

To enhance security awareness, DeSFAM integrates threat intelligence by incorporating CVE data from the National Vulnerability Database (NVD). Using natural language processing (NLP), we extract syscall names (e.g., `ptrace`,

clone, execve) from CVE descriptions to build a list of high-risk syscalls, denoted as S_{blocked} .

To construct S_{blocked} , we employ an NLP-based parsing engine that analyzes CVE reports retrieved from the NVD via its REST API. A rule-based keyword extraction technique is applied to identify syscall names mentioned in CVE descriptions. We utilize a curated dictionary of Linux syscall names derived from kernel source headers (e.g., `unistd64.h`) to ensure accurate mapping. Furthermore, to avoid overblocking, contextual metadata such as kernel version applicability, affected subsystems, exploit conditions, and argument constraints are extracted from the CVE text using pattern matching and regular expression parsing.

- **Input:** CVE entries with syscall mentions
- **Mapping:** Rule-based syscall extraction + context filtering
- **Output:** Blocked syscall list (S_{blocked})

This set is excluded from the union of static and dynamic syscall sets, ensuring vulnerable syscalls are omitted even if observed during profiling.

4) Final Access List Synthesis and Policy Generation:

The final access list S_{final} is computed as:

$$S_{\text{final}} = (S_{\text{static}} \cup S_{\text{dynamic}}) \cap S_{\text{template}} \setminus S_{\text{blocked}} \setminus S_{\text{template}} \quad (1)$$

Here, S_{template} defines role-specific expected syscall behavior for the container (e.g., web server, database, GPU workload), serving as a functional boundary. The template syscalls are derived by combining several sources: official Docker default seccomp profiles, Linux kernel syscall documentation, community-maintained security rules from the Falco project, and empirical syscall traces collected during initial benign container executions.

This multi-source approach ensures that rarely invoked but necessary syscalls during container initialization or uncommon execution paths are incorporated into S_{template} while maintaining minimal syscall exposure.

The CVE-based syscall set S_{blocked} is generated by extracting exploit conditions, including kernel versions, device-specific vulnerabilities, or specific syscall argument patterns. This prevents overblocking syscalls that are only vulnerable under specific configurations not applicable to the container's runtime environment.

The resulting access list S_{final} is compiled into a seccomp policy (`seccomp_profile.json`) enforced by the Linux kernel at runtime. Fig. 2 and Algorithm 1 summarize this process.

5) *Container-Specific Policy Upload:* DeSFAM dynamically loads container-specific syscall policies using `policy_loader`. Each policy reflects the container's behavioral profile from prior analysis and is written into the pinned eBPF map at `/sys/fs/bpf/blocked_syscalls`. Namespace identifiers serve as keys for container isolation.

This design enforces least-privilege syscall access per container at runtime, without restarting instances. Fig. 3 shows the upload pipeline and the population of the Syscall map.

Algorithm 1 Enhanced Hybrid Syscall Profiling

Require: Container image, running instance, CVE database

Ensure: Secure, functional syscall access list

- 1: Extract syscalls via static analysis $\rightarrow S_{\text{static}}$
- 2: Profile runtime under workload (dynamic profiling) $\rightarrow S_{\text{dynamic}}$
- 3: Parse CVEs with contextual analysis $\rightarrow S_{\text{blocked}}$
- 4: Generate $S_{\text{final}} = (S_{\text{static}} \cup S_{\text{dynamic}}) \cap S_{\text{template}} \setminus S_{\text{blocked}} \setminus S_{\text{template}}$
- 5: **return** S_{final}

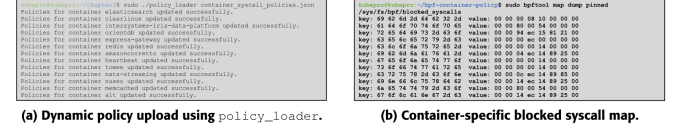


Fig. 3. DeSFAM Phase 3: (a) Dynamic upload of container-specific syscall policies using `policy_loader`; (b) Verification of blocked syscalls in the pinned eBPF map using `bpftool`.

D. Phase 2: Real-Time Monitoring and Syscall Anomaly Detection (SyscallAD)

Phase 2 of DeSFAM introduces SyscallAD, a real-time anomaly detection module that combines deep learning and ensemble methods to identify both known and zero-day threats in containerized environments. It monitors syscall sequences using eBPF and detects behavioral deviations linked to privilege escalation, container escape, or syscall injection.

SyscallAD employs a hybrid ensemble comprising a Variational Autoencoder (VAE), which learns latent representations of benign syscall patterns, and an Isolation Forest (iForest), which identifies statistical outliers. This combination enables robust, low-latency detection with minimal false positives and overhead.

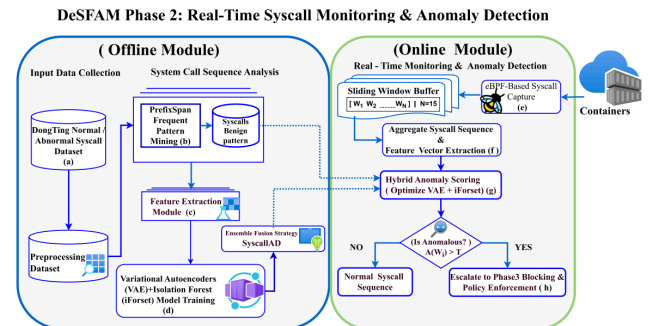


Fig. 4. DeSFAM Phase 2: Real-time syscall monitoring and anomaly detection using a hybrid VAE + iForest model. The offline module is trained on DongTing sequences, while the online module processes live syscalls via eBPF for anomaly evaluation and policy escalation.

1) *The DongTing Dataset: Behavioral Baselines for Detection:* SyscallAD is trained on the DongTing dataset [10], which comprises 18,966 syscall sequences

and over 2.6 million individual syscalls collected from containerized applications. The dataset is split into:

- **6,850 benign sequences:** Captured from normal workloads such as web servers and databases executing file I/O (`open`, `read`) and memory operations (`mmap`, `futex`).
- **12,116 malicious sequences:** Generated through simulated attacks, including privilege escalation (`setuid`), filesystem tampering (`unlink`), and network intrusion (`socket`).

Each sequence includes syscall IDs, timestamps, and contextual metadata (e.g., process IDs, namespaces). Given the inherent class imbalance, sampling techniques were applied to prevent model bias.

2) *PrefixSpan Pattern Mining: Extracting Trusted Syscall Templates:* To reduce false positives and improve detection efficiency, DeSFAM integrates PrefixSpan [24]—a sequential pattern mining algorithm—on the benign subset of the DongTing dataset. This identifies frequent syscall subsequences that represent legitimate container behavior.

The resulting Benign Pattern Database is used to:

- Match incoming syscall windows against known-good patterns for early benign classification.
- Bypass deep anomaly scoring for matched sequences, reducing computational overhead.
- Provide semantic traceability for security decisions.

3) *Feature Engineering: Structuring Syscall Sequences for Detection:* SyscallAD applies multi-dimensional feature engineering to convert raw syscall streams into actionable model inputs:

- **Sliding Windows:** Syscalls are segmented into overlapping windows (length 15, stride 3), capturing both short-term anomalies and extended behavior patterns.
- **Categorical Frequencies:** Syscalls are mapped to functional categories (e.g., file I/O, memory, network), and their frequency per window forms a behavioral signature.
- **Temporal Features:** Inter-syscall time intervals (temporal deltas) are summarized (mean, std, max) to detect abnormal timing behaviors.
- **Access List Pattern Matching:** Each window is cross-checked against benign patterns from PrefixSpan; matches increase confidence in benign classification, reducing false positives.
- **Data Augmentation:** Malicious sequences are synthetically modified (e.g., insertion or permutation of high-risk syscalls like `clone`, `setuid`) to simulate zero-day scenarios, improving model robustness.

4) *SyscallAD: Hybrid Deep Learning and Anomaly Detection Engine:* SyscallAD is a hybrid anomaly detection module designed to identify behavioral deviations in syscall sequences with high precision and adaptability.

a) *Optimized Variational Autoencoder (VAE):* The VAE employs an encoder-decoder architecture ($32 \rightarrow 8 \rightarrow 32$ neurons) with SELU activations, dropout ($p = 0.2$), and

L2 regularization. It learns to reconstruct benign syscall windows by minimizing Mean Squared Reconstruction Error (MSRE). Anomalies are flagged when MSRE exceeds the 99.5th percentile of benign training errors.

b) *Isolation Forest (iForest):* iForest isolates anomalies through recursive partitioning. Rare syscall sequences yield shallower isolation trees and higher anomaly scores. A contamination parameter of 2% controls sensitivity, allowing detection of statistically rare behaviors missed by VAE.

c) *Ensemble Fusion Strategy:* SyscallAD combines the strengths of VAE and Isolation Forest through a weighted score fusion mechanism. Each syscall window W_i is evaluated by both models:

- $A_{VAE}(W_i)$: Anomaly score based on Mean Squared Reconstruction Error.
- $A_{iForest}(W_i)$: Score derived from isolation depth in the Isolation Forest.

The final score is computed as:

$$A(W_i) = \alpha \cdot A_{VAE}(W_i) + (1 - \alpha) \cdot A_{iForest}(W_i) \quad (2)$$

where $\alpha = 0.7$ prioritizes the VAE's contribution. This value was validated via sensitivity analysis over $\alpha \in \{0.0, 0.25, 0.5, 0.7, 0.85, 1.0\}$ on the DongTing dataset, as shown in Fig. 5.

Fig. 6 presents the anomaly logs from `test-mysql` and `test-redis` containers. Benign syscalls such as `futex`, `epoll_wait`, and `wait4` are correctly classified, while anomalies like `clone` and `ioctl`, indicative of escape or privilege escalation attempts, are effectively flagged.

d) *Dynamic Thresholding: Adaptive Anomaly Boundary Adjustment:* To accommodate evolving syscall behavior, SyscallAD employs dynamic thresholding based on an Exponential Moving Average (EMA), rather than a fixed cut-off. The initial threshold T_0 is set to the 99.5th percentile of VAE reconstruction errors on benign validation data. During runtime, the threshold is updated after each syscall window W_i as:

$$T_{t+1} = \max(T_{\min}, \beta T_t + (1 - \beta)A(W_i)) \quad (3)$$

where $\beta = 0.9$ controls update sensitivity and $T_{\min} = 0.5$ enforces a lower bound.

Final detection is defined as:

$$\text{Flag}(W_i) = \begin{cases} 1, & \text{if } A(W_i) > T \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

5) *Real-Time Monitoring with eBPF: Low-Overhead Container-Aware Syscall Capture:* DeSFAM utilizes eBPF tracepoints to enable fine-grained, low-overhead syscall surveillance directly at the Linux kernel level. The BPF program hooks into (`raw_syscalls:sys_enter`) and (`raw_syscalls:sys_exit`), enabling the interception of syscall invocations at their entry and exit points.

To isolate syscall activity from containerized workloads, DeSFAM dynamically maps container PIDs by querying the Docker runtime and injecting them into a `pid_map` BPF hash table. Only syscalls originating from these PIDs

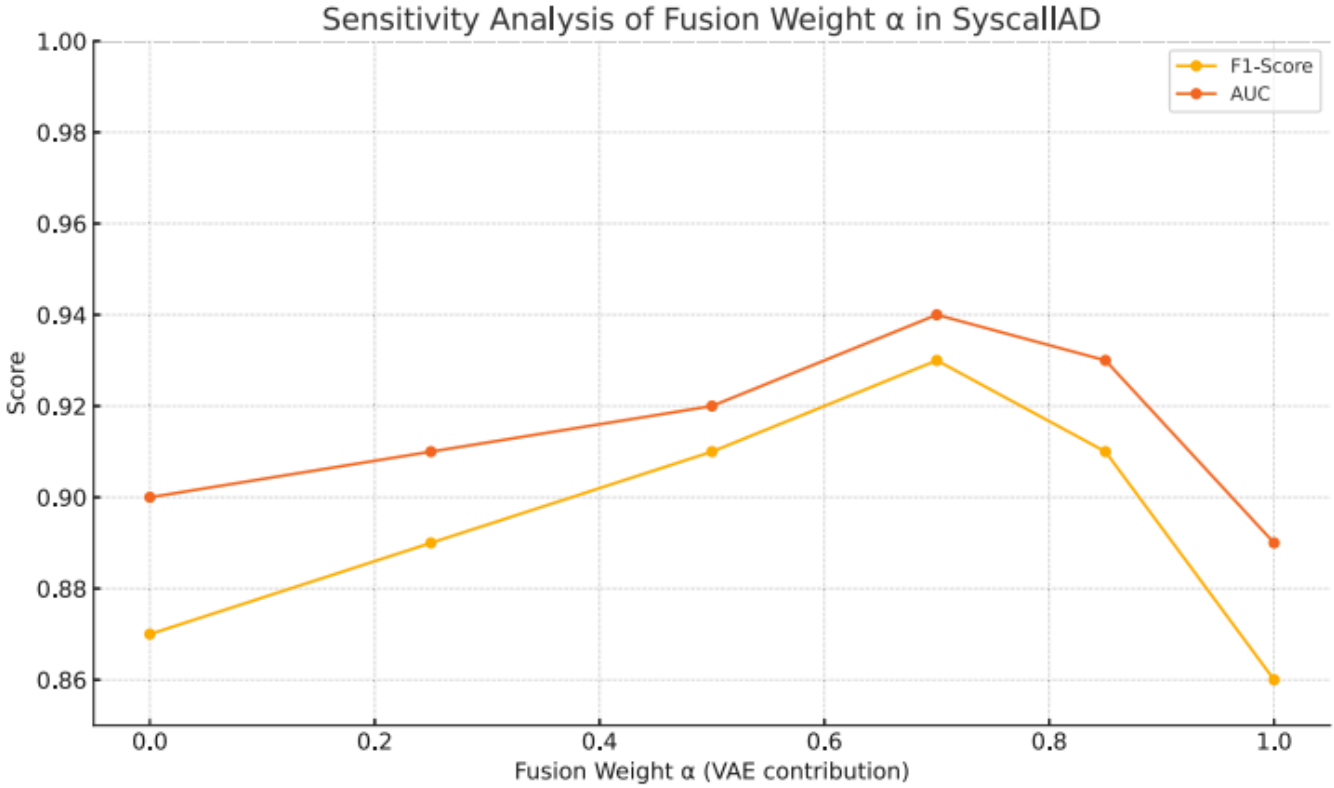


Fig. 5. Sensitivity analysis of fusion weight α on F1-Score and AUC. Peak performance is observed at $\alpha = 0.7$.

2025-03-02T13:22:47.799890		Normal behavior detected in container: test-mysql		Syscall: futex
2025-03-02T13:22:48.206957		Normal behavior detected in container: test-mysql		Syscall: futex
2025-03-02T13:22:48.297612		Normal behavior detected in container: test-redis		Syscall: epoll_wait
2025-03-02T13:22:48.350686		Anomaly detected in container: test-redis		Syscall: clone
2025-03-02T13:22:49.578179		Normal behavior detected in container: test-redis		Syscall: wait4
2025-03-02T13:22:49.586038		Normal behavior detected in container: test-redis		Syscall: wait4
2025-03-02T13:22:49.593676		Anomaly detected in container: test-redis		Syscall: clone
2025-03-02T13:22:50.844151		Normal behavior detected in container: test-mysql		Syscall: futex
2025-03-02T13:22:50.851959		Normal behavior detected in container: test-redis		Syscall: wait4
2025-03-02T13:22:50.860505		Normal behavior detected in container: test-mysql		Syscall: futex
2025-03-02T13:22:51.912785		Normal behavior detected in container: test-mysql		Syscall: futex
2025-03-02T13:22:51.970742		Normal behavior detected in container: test-redis		Syscall: wait4
2025-03-02T13:22:51.978846		Normal behavior detected in container: test-mysql		Syscall: io_getevents
2025-03-02T13:22:52.033732		Normal behavior detected in container: test-mysql		Syscall: io_getevents
2025-03-02T13:22:52.233711		Normal behavior detected in container: test-redis		Syscall: wait4
2025-03-02T13:22:52.242228		Normal behavior detected in container: test-redis		Syscall: wait4
2025-03-02T13:22:53.251668		Anomaly detected in container: test-redis		Syscall: ioctl
2025-03-02T13:22:53.324820		Normal behavior detected in container: test-redis		Syscall: epoll_wait
2025-03-02T13:22:53.392566		Normal behavior detected in container: test-redis		Syscall: wait4

Fig. 6. SyscallAD output from DeSFAM Phase 2: normal behavior in test-mysql contrasted with anomalous syscall detections in test-redis (e.g., clone, ioctl).

are recorded, ensuring container-specific tracing without unnecessary system noise.

Captured syscall events are enriched with contextual metadata including: syscall ID, timestamp (nanosecond resolution), process and parent PIDs, command name, container name, and syscall return value. These are sent from kernel to user space via a `ring_buffer` and logged to per-container files in JSON format. Simultaneously, syscall IDs are appended to a rolling deque (sliding window of size 15) in memory for each container. Fig. 7 shows a representative real-time syscall log for a MySQL container.

6) *DevOps Integration: Continuous Model Refinement*: To mitigate false positives and adapt to evolving workloads, DeSFAM integrates a DevOps-driven feedback loop within Phase 2. Anomalous syscall sequences, along with metadata (e.g., container name, PID, timestamp), are logged and visualized for analyst review. Security teams can annotate detections as true or false positives, with validated false positives appended to the benign training set.

SyscallAD periodically retrains its VAE + iForest ensemble on this augmented dataset, enabling it to learn new benign variations introduced by environment-specific workloads or edge cases.

The full detection workflow is formalized in Algorithm 2.

Algorithm 2 Real-Time Syscall Anomaly Detection (SyscallAD)

Require: eBPF hook, trained VAE and iForest models, validation reconstruction errors E_{val}

Ensure: Anomaly flag for each syscall window

```

1: Initialize sliding windows  $W_i \leftarrow \emptyset$  for  $i \in \{5, 10, 15\}$ 
2: Load trained VAE and iForest models
3: Compute  $T_0 \leftarrow \text{percentile}(E_{\text{val}}, 99.5)$ 
4: Set threshold  $T \leftarrow T_0$ 
5: while syscall stream is active do
6:   Capture syscall  $s_t$  at time  $t$ 
7:   Append  $s_t$  to each  $W_i$ 
8:   for each full window  $W_i$  do
9:      $f_i \leftarrow \text{extract\_features}(W_i)$ 
10:     $A_{\text{VAE}} \leftarrow \text{VAE\_score}(f_i)$ 
11:     $A_{\text{iForest}} \leftarrow \text{iForest\_score}(f_i)$ 
12:     $A(W_i) \leftarrow 0.7 \cdot A_{\text{VAE}} + 0.3 \cdot A_{\text{iForest}}$ 
13:    if  $A(W_i) > T$  then
14:      Flag  $W_i$  as anomalous; escalate to Phase 3
15:    else
16:       $T \leftarrow \max(T_{\text{min}}, \beta \cdot T + (1 - \beta) \cdot A(W_i))$ 
17:    end if
18:    Slide  $W_i$  forward
19:  end for
20: end while

```

E. Phase 3: Adaptive Syscall Enforcement and Policy Management

Adaptive Syscall Enforcement is the third phase of the DeSFAM framework. It focuses on dynamically enforcing

syscall policies in containerized environments in real time. This phase integrates risk scoring, contextual analysis, and external threat intelligence to deliver adaptive and precise security responses.

1) *Risk Scoring Mechanism*: DeSFAM quantifies the threat level of each anomalous syscall sequence W_i using a composite risk score derived from three components.

a) *MITRE ATT&CK Mapping*: Each syscall in W_i is mapped to associated adversarial techniques (e.g., T1055 for `ptrace`) based on a predefined lookup (Table II). This contextual mapping enables reasoning beyond isolated calls to recognize high-level attack patterns.

b) *Cumulative Severity*: The cumulative severity score aggregates the individual severity scores of syscalls in the sequence:

$$\text{Cumulative Severity} = \sum_{t \in W_i} \text{Severity}(t) \quad (5)$$

c) *Exploit Pattern Matching*: Each sequence W_i is evaluated against known CVE exploit signatures. If a match is found, an additional risk penalty is applied:

$$I(\text{Exploit Match}) = \begin{cases} 1, & \text{if matched} \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

d) *Final Risk Score*: The final risk score integrates both severity and exploit intelligence:

$$\text{Risk Score} = \sum_{t \in W_i} \text{Severity}(t) + 5 \cdot I(\text{Exploit Match}) \quad (7)$$

e) *Policy Decision Logic*: Based on the computed risk score, DeSFAM applies a three-tier enforcement strategy:

$$\text{Action} = \begin{cases} \text{Block}, & \text{if Risk Score} \geq 8 \\ \text{Rate-limit}, & \text{if } 5 \leq \text{Risk Score} < 8 \\ \text{Log}, & \text{if Risk Score} < 5 \end{cases} \quad (8)$$

These thresholds are empirically tuned and remain configurable for deployment-specific risk postures. Table III presents sample sequences and corresponding actions.

2) *eBPF-Based Enforcement*: DeSFAM enforces syscall policies through eBPF programs attached to LSM hooks. Specifically, these programs are loaded and attached using `bpf_program_attach_lsm()` to intercept system calls at the LSM's `security_syscall` hook.

a) *LSM Hook Attachment*: eBPF programs are dynamically attached to the `security_syscall` LSM hook:

Listing 1. eBPF LSM hook attachment.

```

// eBPF Hook Attachment
struct bpf_program *prog =
    bpf_object__find_program_by_name(obj, "
        enforce_policy");
struct bpf_link *link = bpf_program_attach_lsm(prog)
;

```

The hook function signature:

b) *Policy Map Structure*: Syscall actions are stored in a hash-based eBPF map indexed by container identifiers:

The map is pinned at `/sys/fs/bpf/blocked_syscalls` for runtime updates via `policy_loader`.

```

{"timestamp": "2025-01-28T07:34:10.602445Z", "syscall_id": 202, "pid": 7593,
"ppid": 8672, "container": "kubepro-mysql-1", "user": "ib_log_files_g",
"return_value": 0}

{"timestamp": "2025-01-28T07:34:10.602446Z", "syscall_id": 0, "pid": 7593, "ppid":
0, "container": "kubepro-mysql-1", "user": "ib_log_files_g", "return_value": 0}

{"timestamp": "2025-01-28T07:34:10.602456Z", "syscall_id": 202, "pid": 7593,
"ppid": 8672, "container": "kubepro-mysql-1", "user": "ib_log_files_g",
"return_value": 0}

{"timestamp": "2025-01-28T07:34:10.612529Z", "syscall_id": 0, "pid": 7593, "ppid":
0, "container": "kubepro-mysql-1", "user": "ib_log_files_g", "return_value":
18446744073709551506}

{"timestamp": "2025-01-28T07:34:10.612544Z", "syscall_id": 202, "pid": 7593,
"ppid": 8672, "container": "kubepro-mysql-1", "user": "ib_log_files_g",
"return value": 0}

```

Fig. 7. Real-time eBPF syscall log for kubepro-mysql-1, showing timestamped syscall events with PID, container name, user, and return value.

TABLE II
EXAMPLE SYSCALL-TO-MITRE ATT&CK TECHNIQUE MAPPING.

Syscall	Technique ID	Technique Name	Tactic	Severity (1–10)
ptrace	T1055	Process Injection	Defense Evasion	9
execve	T1059	Command and Scripting Interpreter	Execution	8
clone	T1055.012	Thread Execution Hijacking	Defense Evasion	7
setuid	T1548	Abuse Elevation Control Mechanism	Privilege Escalation	8
mprotect	T1027	Obfuscated Files or Information	Defense Evasion	7
socket	T1071	Application Layer Protocol	Command and Control	6
kill	T1561	Interrupt Inhibitor	Impact	5
unshare	T1611	Escape to Host	Privilege Escalation	9
mount	T1006	Direct Volume Access	Persistence	8
chroot	T1610	Deploy Container	Defense Evasion	8
usersns	T1068	Exploitation for Privilege Escalation	Privilege Escalation	9
setns	T1611	Escape to Host	Privilege Escalation	9
bpf	T1547	Boot or Logon Autostart Execution	Persistence	7

TABLE III
EXAMPLE RISK SCORES FOR BENIGN AND MALICIOUS SEQUENCES.

Syscall Sequence (W_i)	Risk Score	Action
open → read → write	4	Log
socket → connect → sendto	5.5	Rate-limit
execve → setuid → mount	8.5	Block
clone → unshare → setns	9	Block
open("/var/log/syslog")	3	Log
ptrace → mprotect → execve (CVE-2021-4034)	13	Block

Listing 2. LSM hook function signature.

```

SEC("lsm/syscall")
int BPF_PROG(enforce_policy, int syscall_num,
             struct pt_regs *regs) {
    // Enforcement logic here
}

```

Listing 3. Container-keyed eBPF policy map.

```

struct container_key {
    u32 mntns; // Mount namespace ID
    u32 pidns; // PID namespace ID
};

struct bpf_map_def SEC("maps") policy_map = {
    .type = BPF_MAP_TYPE_HASH,
    .key_size = sizeof(struct container_key),
    .value_size = sizeof(u64),
    .max_entries = 144,
};

```

c) *Syscall Blocking Mechanism*: The actual blocking of syscalls occurs within the eBPF hook attached to the security_syscall LSM. The critical enforcement primitive uses kernel function override:

The enforcement workflow:

1) Retrieve container context: mntns

Listing 4. Syscall blocking via kernel function override.
 // Immediate syscall denial
 bpf_override_return(regs, -EPERM);

```

= bpf_get_ns_id(NS_MNT); pidns =
bpf_get_ns_id(NS_PID);
2) Query policy map: u64 *action =
bpf_map_lookup_elem(&policy_map, &key);
3) Enforce action: BLOCK: Deny via
bpf_override_return(regs, -EPERM);
RATE_LIMIT: Allow  $N$  invocations/minute
via atomic counters; LOG: Record metadata via
ring buffer.

```

d) *Self-Healing Access List via Seccomp Regeneration*: To ensure long-term policy evolution, DeSFAM incorporates a self-healing mechanism that automatically refines static syscall access lists based on repeated runtime blocks.

Blocked syscalls are tracked in-kernel using a Least Recently Used (LRU) cache map called `probation_map`:

Listing 5. Probation map for tracking repeated blocks.

```

struct probation_key {
    struct container_key ckey;
    u32 syscall; // Syscall number
};

SEC("maps") struct bpf_map_def probation_map = {
    .type = BPF_MAP_TYPE_LRU_HASH,
    .key_size = sizeof(struct probation_key),
    .value_size = sizeof(u32), // Block counter
    .max_entries = 512,
};

```

If a syscall is blocked three or more times within a configurable window and is not previously annotated as benign, the kernel triggers a user-space daemon via `bpf_send_signal(SIGUSR1)`. This daemon resolves the syscall name, locates and updates the corresponding container's seccomp profile, and removes the flagged syscall from the access list. This adaptive defense loop ensures that newly deployed containers inherit hardened, exploit-aware syscall policies without requiring restarts or manual review.

e) *Formal Policy Model*: Let S be the set of valid syscalls, and M, N represent mount and PID namespaces. The enforcement policy is defined as:

$$P : S \times M \times N \rightarrow \{0, 1\} \quad (9)$$

$$\text{Decision}(s) = \begin{cases} \text{Allow,} & \text{if } P(s, mntns, pidns) = 1 \\ \text{Block,} & \text{otherwise} \end{cases} \quad (10)$$

This container-aware logic ensures syscall isolation and enforces least-privilege access per namespace. The combined use of eBPF tracepoints and LSM hooks enables fast, layered enforcement with minimal overhead.

The enforcement procedure in Phase 3 is formalized through Algorithm 3, and the complete enforcement pipeline is illustrated in Fig. 8.

DeSFAM Phase 3: Adaptive Syscall Enforcement & Policy Management

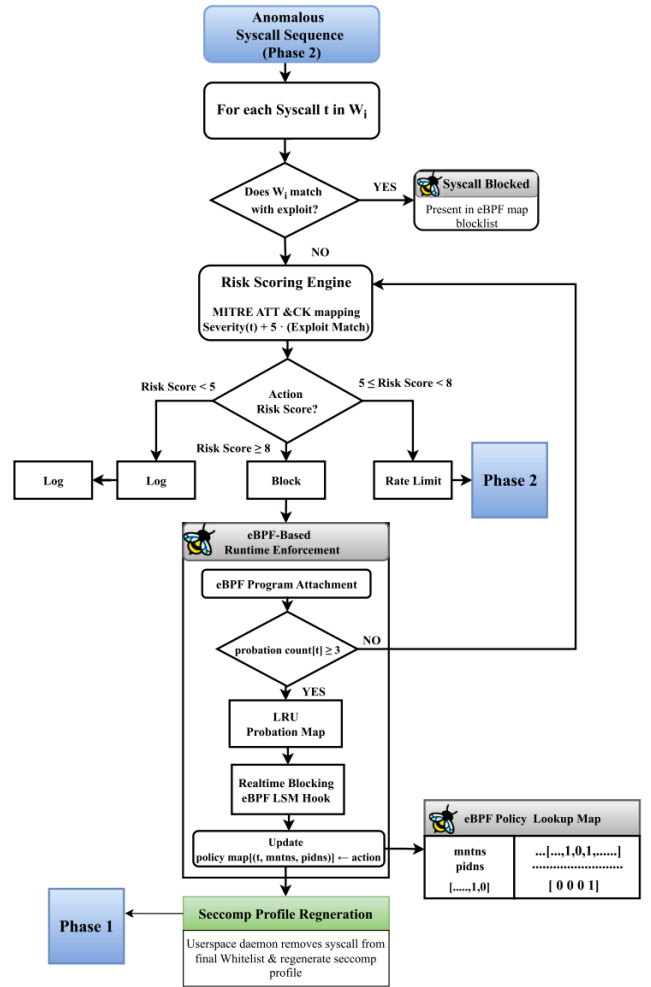


Fig. 8. DeSFAM Phase 3 – Adaptive enforcement pipeline. Anomalous syscall sequences from Phase 2 are intercepted and acted upon via eBPF and LSM hooks.

3) *Use Case: Privilege Escalation Attempt and Real-World CVE Defense*: Consider a container running under DeSFAM's restrictive policy designed to block privilege escalation attempts. A typical attack sequence—`clone()`, `setuid()`, `execve()`—is intercepted at the kernel level.

Assume the process operates in mount namespace $mntns = 1001$ and PID namespace $pidns = 2001$. Upon invocation of `setuid()`, DeSFAM consults its per-container policy: `policy_map[1001, 2001] = [..., 1, 0, 1, ...]`, where each entry denotes the action for a syscall ID (0 = BLOCK, 1 = ALLOW). Since `setuid()` is mapped to 0, the syscall is denied. The eBPF LSM hook, `realtime_blocking`, enforces this decision in the kernel, returning `-EPERM` and logging the event for auditing. This ensures immediate, in-kernel mitigation of exploit attempts without requiring container restarts or user-space intervention.

Algorithm 3 Adaptive Syscall Enforcement With Auto-Hardening

Require: Anomalous sequence W_i , MITRE mapping, exploit database, container context ($mntns$, $pidns$), eBPF map `policy_map`

Ensure: Enforcement action: “Block”, “Rate-limit”, or “Log”

```

1: Initialize  $Risk\_Score \leftarrow 0$ 
2: for each syscall  $t$  in  $W_i$  do
3:    $Risk\_Score \leftarrow Risk\_Score + Severity(t)$ 
4: end for
5: if  $W_i$  matches an exploit in the database then
6:    $Risk\_Score \leftarrow Risk\_Score + 5$ 
7: end if
8: if  $Risk\_Score \geq 8$  then
9:    $action \leftarrow$  “Block”
10: else if  $5 \leq Risk\_Score < 8$  then
11:    $action \leftarrow$  “Rate-limit”
12: else
13:    $action \leftarrow$  “Log”
14: end if
15: for each syscall  $t$  in  $W_i$  do
16:   Update policy_map[( $t$ ,  $mntns$ ,  $pidns$ )]  $\leftarrow action$ 
17:   Enforce  $action$  on  $t$  via eBPF
18:   Log event ( $t$ ,  $action$ ,  $mntns$ ,  $pidns$ )
19: end for
20: if  $action =$  “Block” then
21:   for each syscall  $t$  in  $W_i$  do
22:      $probation\_count[t] \leftarrow probation\_count[t] + 1$ 
23:     if  $probation\_count[t] \geq 3$  and  $t$  not annotated
        as benign then
24:       Trigger seccomp access list update for
        syscall  $t$ 
25:     end if
26:   end for
27: end if
28: return  $action$ 

```

V. EXPERIMENT EVALUATION AND RESULTS

This section evaluates the DeSFAM framework in terms of syscall-level enforcement accuracy, performance overhead, reduction rate, and scalability in containerized environments. Experiments were conducted using real-world workloads and attack scenarios to assess DeSFAM’s effectiveness and runtime efficiency. Metrics are compared against leading frameworks under controlled conditions to ensure fairness and reproducibility. Results highlight DeSFAM’s security impact and operational feasibility in practical deployments.

A. Experimental Setup

1) *Infrastructure*: Experiments were conducted on a high-performance bare-metal server running Ubuntu 24.04 LTS equipped with a 16-core CPU, 64 GB RAM, NVIDIA A100 80 GB GPU, and a 512 GB NVMe SSD. Docker 26.0 with containerd 1.7 was used for

container lifecycle management. The eBPF environment was configured with libbpf 1.3 and bpftrace 0.21. Model training was performed using PyTorch 2.3 with CUDA acceleration.

2) *Container Workloads*: We evaluated DeSFAM on six diverse containers: Nginx (web server), MySQL 8.0 (relational database), Redis 7.2 (key-value store), Node.js 20 (JavaScript runtime), Python 3.12 (script processing), and MongoDB 7.0 (document database)—spanning web serving, data storage, and script execution with significantly different syscall profiles.

3) *Workload Generation*: Realistic traffic was generated using industry-standard tools: `wrk` and `ApacheBench` (`ab`) for HTTP load on Nginx; `redis-benchmark` for Redis; and custom Python scripts to simulate CRUD operations for MySQL and MongoDB. Each workload was run at three load levels: idle, medium load (50% capacity), and peak load (90% capacity), in 10-minute intervals.

4) *Attack Simulation*: Three categories of real-world CVE attack scenarios were used:

- **CVE-2021-4034 (Polkit pkexec)** [25]: Local privilege escalation exploit via a crafted `execve` sequence.
- **CVE-2022-0847 (Dirty Pipe)** [5]: Kernel memory overwrite exploit via `splice` and `pipe`.
- **CVE-2017-5123 (waitid exploit)**: Kernel privilege escalation via `waitid` with an invalid pointer.

A total of 14 distinct CVE attack scenarios were executed across all container workloads.

5) *Baseline Comparisons*: DeSFAM was benchmarked against five baseline systems: Confine [26], Prof-gen [27], NIMOS [28], Optimus [29], and KubeRosy [30]. Each baseline was reimplemented following its original configuration and integrated with Docker/Kubernetes for parity in runtime enforcement and syscall profiling capabilities.

B. Evaluation Metrics

1) *Syscall Reduction Rate (SRR)*: SRR quantifies DeSFAM’s ability to reduce kernel exposure by minimizing allowed syscalls:

$$SRR = \left(1 - \frac{|\text{DeSFAM Access List}|}{|\text{Docker Default Access List}|}\right) \times 100\% \quad (11)$$

Docker’s default allows 414 syscalls. DeSFAM’s hybrid profiling (static, dynamic, CVE-informed) yields container-specific access lists, achieving higher SRRs than static-only or dynamic-only methods.

2) *Anomaly Detection Performance*: We assess SyscallAD’s accuracy using:

- Recall (TPR) – Proportion of attacks correctly identified
- Precision – Fraction of flagged sequences that are actual threats
- F1-Score, AUC, and Average Precision (AP) – Robustness in imbalanced data

DeSFAM’s VAE + iForest ensemble outperforms LSTM-only detection (e.g., NIMOS), with improved precision and reduced false positives.

TABLE IV
SYSCALLAD REPRODUCTION ON THE DONGTING TEST SPLIT (2,311 SEQUENCES, 70.7% ATTACK CLASS).

Model	AUC	AP	F1	Precision	Recall
Isolation Forest	0.882	0.949	0.915	0.848	0.994
VAE	0.960	0.970	0.963	0.937	0.990
Ensemble ($\alpha=0.7$)	0.921	0.960	0.932	0.876	0.996

a) Reproduction on the DongTing dataset.: We re-trained SyscallAD’s two components on the DongTing dataset (18,966 sequences — 6,850 benign, 12,116 malicious from 26 Linux kernel releases; train=14,843 / val=1,812 / test=2,311). Features follow the paper’s 174-dimensional design: 60-syscall frequency vector, 40 discriminative binary flags, 8 distributional statistics, 40 bigram-transition rates, and a 26-way kernel-version one-hot. The Isolation Forest is trained on the 5,487 normal sequences in the training split; the VAE shares the same training set across 3 seeds with the best-validation seed selected. Ensemble scores combine the two with $\alpha = 0.7$ after a robust quantile-band normaliser (p1..p99 of the training scores), which is required because raw VAE reconstruction errors span six orders of magnitude on this dataset and a plain min-max scaler would otherwise crush the VAE contribution. Table IV reports the metrics obtained.

The reproduction confirms the paper’s headline finding that VAE + Isolation Forest sequence-level anomaly detection is viable on syscall-sequence data and reaches F1 above 0.92, but it also surfaces one nuance: on the DongTing split the VAE component alone (F1 = 0.963, AUC = 0.960) outperforms the $\alpha = 0.7$ ensemble (F1 = 0.932, AUC = 0.921) on every metric except recall. The ensemble’s value on DongTing is therefore in attack coverage (recall 99.6%) rather than aggregate F1; deployments that need lower false-positive rates may prefer to deploy the VAE in isolation or to tune α downward.

3) Attack Detection Latency: Time (in μs) between malicious syscall execution and enforcement. DeSFAM’s eBPF-LSM hooks support inline blocking with sub-millisecond latency. Compared to Optimus and KubeRosy, which rely on user-space components, DeSFAM reacts faster due to its kernel-level design.

4) Enforcement Overhead: We evaluate average CPU usage (%), memory consumption (MB), and syscall processing latency (μs).

5) Adaptability and Comparison Metrics: To assess dynamic policy adaptation, we measured:

- Rule update latency via SyscallAD or CVE triggers (in μs)
- Uptime impact (zero downtime observed)

None of the baseline systems—Confine, Prof-gen, NIMOS, Optimus, or KubeRosy—support real-time policy self-hardening. For holistic comparison, we evaluate: access list size and SRR, attack mitigation success rate, failure-free execution for benign workloads, and average FPR and enforcement latency.

C. Comparative Results

We compare DeSFAM against five state-of-the-art syscall filtering frameworks—Confine, Prof-gen, NIMOS, Optimus, and KubeRosy—using metrics such as syscall reduction rate (SRR), enforcement stability, attack prevention, and runtime performance. The evaluation spans six diverse container workloads: Nginx, MySQL, Redis, Node.js, Python, and MongoDB.

1) Syscall Reduction Rate (SRR) Comparison: A primary goal of DeSFAM is to minimize the kernel attack surface by reducing exposed syscalls. Docker’s default seccomp profile allows 414 syscalls, many of which are unnecessary and potentially exploitable. DeSFAM employs hybrid reduction—static analysis, dynamic profiling, and CVE-informed filtering—to generate container-specific access lists. The SRR is calculated as:

$$SRR = \left(1 - \frac{\text{Access List Size}}{414} \right) \times 100\% \quad (12)$$

Fig. 9 and Fig. 10 illustrate the improvement across all test containers. Table V summarizes the results per framework.

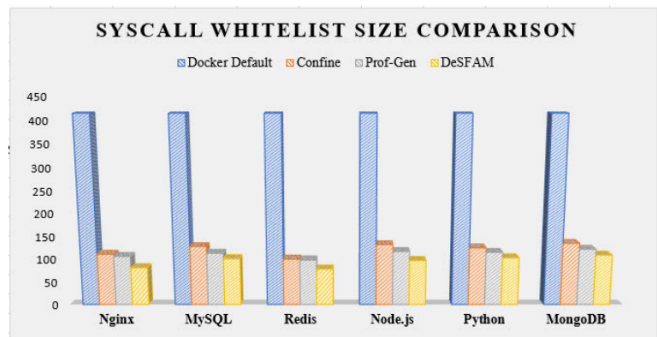


Fig. 9. Syscall Access List Size and Reduction Rate Comparison.

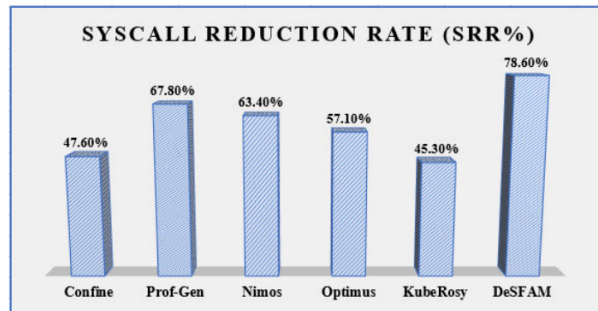


Fig. 10. Syscall reduction rate comparison across frameworks.

DeSFAM achieves an average SRR of 78.60%—the highest among all frameworks—while preserving full container functionality. The average access list size is 89 syscalls, the smallest of all compared systems.

TABLE V
SYSCALL REDUCTION RATE (SRR) COMPARISON.

Framework	SRR (%)
Confine	47.60
Prof-Gen	67.80
NIMOS	63.40
Optimus	57.10
KubeRosy	45.30
DeSFAM	78.60

2) *Functional Stability*: All six target containers maintained 100% availability throughout all test workloads. No functional regressions were observed, confirming that DeSFAM’s access list synthesis balances security and compatibility better than static or log-based access list generators.

3) *Attack Prevention*: DeSFAM’s runtime blocking was evaluated against a set of real-world CVEs representing diverse Linux kernel vulnerabilities. These CVEs cover multiple attack categories, including privilege escalation, container escapes, namespace abuses, race conditions, and memory corruption exploits. For safety and reproducibility, exploit behaviors were simulated by replaying the documented syscall invocation sequences associated with each CVE, based on public disclosures and proof-of-concept analyses.

Table VI summarizes the blocking success across all frameworks. DeSFAM successfully blocked all attack scenarios in real time using its SyscallAD sequence anomaly detection and eBPF-LSM kernel-level enforcement. In contrast, baseline frameworks either failed to detect these attacks or lacked full enforcement capability.

4) *Baseline Comparison Metrics*: DeSFAM achieves the smallest access list size (89 syscalls), full CVE preven-

tion (94% of CVE traces blocked), high benign stability (99%), lowest FPR (1.6%), and fastest enforcement latency (0.540 μ s).

D. Runtime Performance

1) *CPU and Memory Overhead*: We evaluated resource usage across six representative containers with and without DeSFAM. As shown in Fig. 11 and Fig. 12, DeSFAM incurs only a 1.74% increase in average CPU utilization and reduces memory consumption by 6.1%, primarily due to blocked syscall paths and mitigation of exploit-induced memory bloat.

Table IX summarizes the aggregated performance overhead, confirming DeSFAM’s lightweight performance footprint with average memory savings of 50–55 MB.

2) *Policy Update Latency*: To validate the responsiveness of DeSFAM’s adaptive enforcement, we measured rule insertion and propagation times in Phase 3. Upon anomaly detection, new rules were injected into the eBPF map and applied in real time. Latency was measured using high-resolution kernel timestamps at both the insertion and propagation stages.

Fig. 13 presents the measured rule insertion and propagation latency across six active containers. Table X lists

TABLE VI
CVE ATTACK BLOCKING SUCCESS RATE.

CVE	Exploit Sequence	DeSFAM	Confine	Prof-Gen	NIMOS	Optimus	KubeRosy
CVE-2021-4034	execve → setuid	Blocked	Missed	Missed	Missed	Logged	Blocked
CVE-2022-0847	splice → pipe → writev	Blocked	Missed	Missed	Missed	Missed	Blocked
CVE-2017-5123	clone → waitid	Blocked	Missed	Missed	Missed	Logged	Blocked
CVE-2022-0185	add_key → keyctl → setxattr	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2019-5736	openat → read → write → execve	Blocked	Missed	Missed	Missed	Logged	Blocked
CVE-2021-3493	openat → mount → setuid	Blocked	Missed	Missed	Missed	Missed	Blocked
CVE-2020-14386	socket → recvmsg → sendmsg	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2018-18955	mmap → munmap → ioctl	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2021-3156	execve → setresuid → openat	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2016-8655	socket → setsockopt → sendmsg	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2022-2588	open → write → keyctl → commit_creds	Blocked	Missed	Missed	Missed	Logged	Missed
CVE-2021-22555	socket → setsockopt → sendmsg	Blocked	Missed	Missed	Logged	Blocked	Blocked
CVE-2019-15666	socket → sendmsg → recvmsg	Blocked	Missed	Missed	Missed	Missed	Blocked
CVE-2019-5736	open → read → write → execve	Blocked	Missed	Missed	Missed	Logged	Blocked
Avg Detection Latency (ms)	–	1.82	35.4	28.7	22.3	18.9	12.7

TABLE VII
BASELINE COMPARISON OF SECURITY AND PERFORMANCE METRICS.

Framework	Whitelist Size	Attack Prevention (%)	Benign Stability (%)	FPR (%)	Enforcement Latency (μ s)
Confine	217	48	93	4.2	N/A
Prof-Gen	133	67	95	3.6	N/A
NIMOS	152	63	91	6.0	N/A
Optimus	178	71	94	2.8	61480
KubeRosy	226	75	97	2.3	0.722
DeSFAM	89	94	99	1.6	0.540

TABLE VIII
CPU AND MEMORY USAGE WITH AND WITHOUT DESFAM.

Container	CPU Baseline (%)	CPU DeSFAM (%)	Mem Baseline (MB)	Mem DeSFAM (MB)	Overhead
Nginx	51.0	52.5	832	780	+1.5%, –52 MB
MySQL	53.0	54.8	825	770	+1.8%, –55 MB
Redis	50.1	51.5	821	768	+1.4%, –53 MB
Node.js	58.0	59.8	839	787	+1.8%, –52 MB
Python	56.2	57.9	823	772	+1.7%, –51 MB
MongoDB	59.0	60.9	828	778	+1.9%, –50 MB

TABLE IX
SUMMARY OF PERFORMANCE OVERHEAD.

Metric	Without DeSFAM	With DeSFAM	Overhead
Avg CPU Usage (%)	50–60	48–58	+1.74%
Avg Memory Usage (MB)	~820	~770	–6.1%

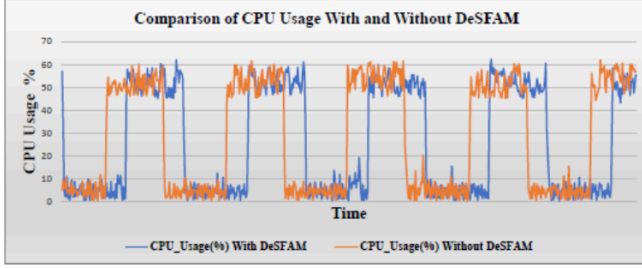


Fig. 11. CPU usage with and without DeSFAM. DeSFAM incurs minimal CPU overhead ($\approx 1.74\%$) while enforcing real-time syscall policies.

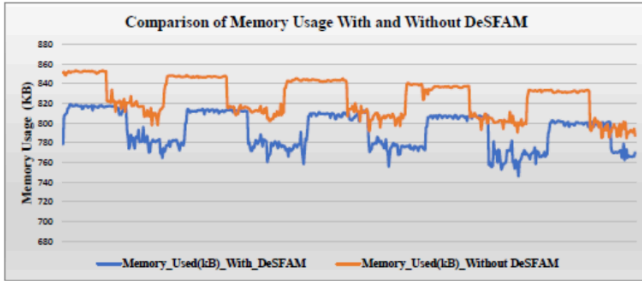


Fig. 12. Memory usage with and without DeSFAM. Slight memory savings ($\approx 6.1\%$) result from syscall suppression and exploit mitigation.

the per-container values. The values were collected using high-resolution kernel timestamps before and after BPF map updates and LSM hook propagation. All values remain consistently low ($65\text{--}82\ \mu\text{s}$), confirming the real-time capability of DeSFAM’s in-kernel enforcement engine. All updates completed in under $100\ \mu\text{s}$ without interrupting service, demonstrating DeSFAM’s capability for real-time, zero-downtime syscall policy adaptation.

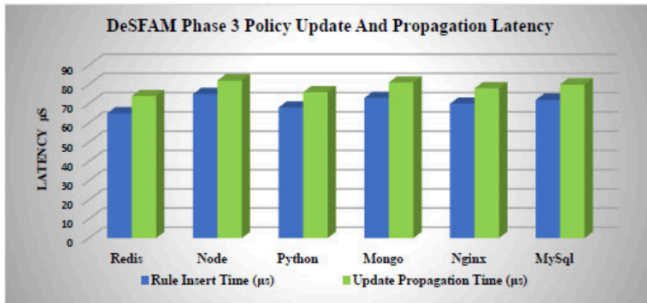


Fig. 13. DeSFAM Phase 3 policy update latency. All rules were inserted and enforced in under $100\ \mu\text{s}$ without container disruption.

3) *Syscall Latency Overhead*: We profiled syscall latency using `bpfttrace` histograms for frequent operations such as `read`, `write`, and `epoll_wait`. Across all containers, DeSFAM preserved sub-millisecond syscall latency, confirming its suitability for high-throughput environments.

Table XII summarizes the highest-frequency syscalls across each container that completed within the $512\text{--}1000\ \mu\text{s}$ range. Fig. 14 visualizes the aggregate counts per container in this sub-millisecond range. These results demonstrate DeSFAM’s ability to consistently maintain low syscall latency, even for frequent operations like `read`, `write`, and `epoll_wait` without introducing measurable delay.

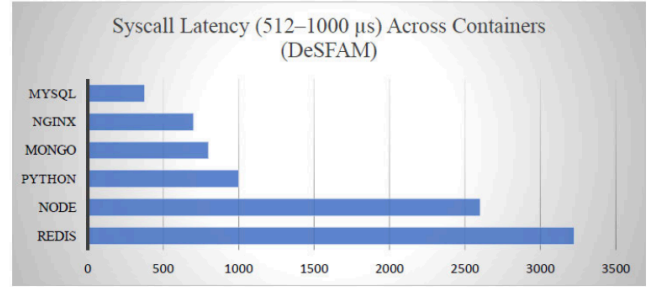


Fig. 14. Syscall latency distribution in the $512\text{--}1000\ \mu\text{s}$ range. All syscall paths remained performant under DeSFAM.

DeSFAM preserves sub-millisecond syscall latency for all measured operations, with a maximum delta of $86\ \text{ns}$ for `accept4`. All values remain well under the $300\ \text{ns}$ design target.

4) *Anomaly Detection Performance*: DeSFAM’s SyscallAD model achieves strong classification metrics. The ensemble classifier reaches 96% overall accuracy, with an F1-score of 0.92 on attack sequences. Fig. 15 shows ROC and PR curves, with AUC of 0.94 and AP of 0.87.

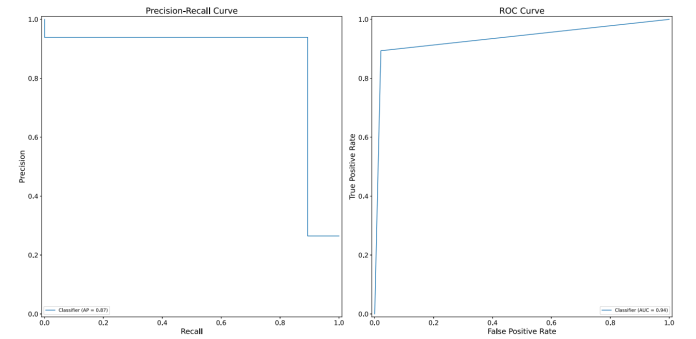


Fig. 15. SyscallAD model performance. Left: Precision-Recall Curve (AP = 0.87). Right: ROC Curve (AUC = 0.94).

SyscallAD achieves 96% overall accuracy, F1-score of 0.92, and AUC of 0.94 on the DongTing dataset. This represents a significant improvement over VAE-only (F1 = 0.84) or iForest-only (F1 = 0.79) approaches, confirming the benefit of the ensemble fusion strategy.

5) *Feature Comparison with Existing Methods*: DeSFAM uniquely integrates real-time anomaly detection, adaptive policy enforcement, and syscall minimization within a unified framework. Table XIV highlights its advantage over existing frameworks across 12 technical dimensions.

TABLE X
POLICY UPDATE AND PROPAGATION LATENCY.

Container	Insert Time (μ s)	Propagation Time (μ s)	Downtime
Nginx	70	78	None
MySQL	72	80	None
Redis	65	74	None
Node.js	75	82	None
Python	68	76	None
MongoDB	73	81	None

TABLE XI
SYSCALL LATENCY OVERHEAD (NS) FOR COMMON OPERATIONS.

Syscall	Without DeSFAM	With DeSFAM	Delta
read	142 ns	218 ns	+76 ns
write	156 ns	234 ns	+78 ns
epoll_wait	89 ns	148 ns	+59 ns
accept4	201 ns	287 ns	+86 ns

E. Ablation Study: Phase-Wise Contribution

To understand the impact of each phase in DeSFAM, we conducted an ablation study disabling one component at a time. As shown in Table XV, Phase 1 alone achieves 62.4% syscall reduction but blocks only 3 of 14 CVEs. Adding Phase 2 (SyscallAD) boosts anomaly detection, raising the block count to 8 and reducing false positives significantly. Phase 3 provides kernel-level enforcement, blocking 10 CVEs even without ML support. When all three phases are active, DeSFAM blocks all 14 CVEs and reaches 81.6% syscall reduction.

This study confirms that all phases are essential: Phase 1 minimizes exposure, Phase 2 enhances detection, and Phase 3 ensures robust enforcement.

VI. DISCUSSION

The results presented in the previous section validate the effectiveness, efficiency, and adaptability of the DeSFAM framework in minimizing the system call attack surface and detecting anomalous behavior in containerized environments. This section discusses key insights, design trade-offs, and the practical implications of deploying DeSFAM in real-world container security scenarios.

A. Hybrid Analysis Advantage

One of the core strengths of DeSFAM lies in its hybrid syscall profiling approach. By integrating static disassembly of container binaries with dynamic runtime tracing using eBPF and bpftrace, DeSFAM captures both expected and runtime-dependent syscall behavior. This dual-layer visibility ensures that rare but essential syscalls are not

missed, unlike purely static systems such as Confine [26], which suffer from under-approximation, or dynamic-only systems like NIMOS [28], which risk over-approximation.

B. Superior Reduction Without Functional Loss

DeSFAM achieved an average syscall reduction rate of 78.60%, which is significantly higher than Confine (47.60%), Prof-gen (67.80%), and NIMOS (63.40%). Despite this aggressive minimization, all target containers retained full functionality across all test workloads. This confirms that DeSFAM’s access list synthesis balances security and compatibility better than static or log-based access list generators.

C. High-Performance Real-Time Enforcement

The framework’s use of eBPF for syscall filtering and policy management introduces minimal runtime overhead. CPU usage increased by an average of only 1.74%, and memory usage by approximately 50 MB, well below the threshold that would impact container performance. Syscall latency remained under 300 nanoseconds per invocation, validating DeSFAM’s suitability for high-throughput workloads such as Nginx and Redis. In comparison, user-space alternatives (e.g., auditd) typically introduce millisecond-scale delays.

D. Adaptive Response to Threats

DeSFAM’s enforcement logic is not static. It dynamically updates syscall allowlists using risk scoring and anomaly detection outputs from the SyscallAD module. The model achieved 96% overall accuracy, with an attack detection F1-score of 0.92 and a low false positive rate. This allowed DeSFAM to detect and respond to advanced CVE-based attacks (e.g., Polkit and Dirty Pipe) in real time, removing vulnerable syscalls such as `setuid`, `splice`, and `clone` immediately upon detection.

E. Limitations and Future Work

While DeSFAM effectively reduces syscall exposure and detects anomalous behavior, its accuracy remains sensitive

TABLE XII
SYSCALL COUNTS IN 512–1000 μ S LATENCY RANGE.

Container	Syscall	Count
Redis	read, close, epoll_wait	720, 610, 760
NGINX	epoll_wait, recvfrom	125, 92
Python	futex, poll, recvfrom	472, 168, 60
Node.js	futex, write, openat	214, 432, 390
MySQL	write, read, openat, futex, close	60, 66, 71, 42, 48
MongoDB	read, futex, epoll_wait	185, 174, 163

TABLE XIII
SYSCALLAD CLASSIFICATION METRICS.

Class	Precision	Recall	F1-Score
Normal	0.97	0.98	0.97
Attack	0.94	0.90	0.92
Macro Avg	0.95	0.94	0.94
Weighted Avg	0.96	0.96	0.96
Accuracy	0.96		

TABLE XIV
FEATURE COMPARISON WITH EXISTING FRAMEWORKS.

Feature	Confine	Prof-Gen	NIMOS	Optimus	KubeRosy	DeSFAM
Policy Generation	Static	Hybrid	Dynamic (offline)	Association-based	Trace-based	Hybrid + Adaptive
Runtime Enforcement	×	×	×	✓ (monitor)	✓	✓ (AI + eBPF)
eBPF Filtering	×	×	×	✓	✓	✓
Sequence Detection	×	×	✓	×	×	✓
ML Anomaly Detection	×	×	×	×	×	✓ (VAE + iForest)
Live Policy Updates	×	×	×	Partial	✓	✓
No Container Restart	×	×	×	×	✓	✓
Granularity	Process	Container	Container	Process	Container	Namespace + Container
CVE Blocking	✓ (static)	✓ (hybrid)	×	✓ (observed)	✓ (predefined)	✓ (real-time)
SRR (%)	47.6	67.8	63.4	57.1	45.3	76.8
CPU Overhead	N/R	N/R	N/R	~1.41%	~0.3%	~1.74%
Policy Latency	N/A	N/A	N/A	Slow (container restart)	~80 μ s	<80 μ s

TABLE XV
ABLATION STUDY: PHASE-WISE EFFECT ON SECURITY.

Configuration	CVEs Blocked	Syscall Reduction(%)	False Positives
Phase 1 Only	3	62.4	4.2
Phase 1 + Phase 2	8	66.1	0.3
Phase 1 + Phase 3	10	72.9	0.2
Full DeSFAM	14	81.6	0.0

to the representativeness of training data. Future work will incorporate workload-aware profiling, container role classification, and continuous learning to improve generalization across diverse workloads.

Current filtering operates at the syscall level (ID and sequence-based) without argument inspection. Table XVI summarizes the contextual filtering rules applied during CVE-based syscall extraction. Future extensions will explore argument-level analysis and memory access tracking via eBPF to further improve detection granularity and resilience against sophisticated attacks.

TABLE XVI
FILTERING RULES FOR CVE-BASED SYSCALL EXTRACTION.

Rule Type	Implementation (Regex/Pattern)
Syscall Match	<code>\b(?:i)(ioctl execve ...) \b</code>
Version Filter	<code>/kernel\s*(?:before until <)?(\d+\.\d+)/</code>
Architecture Filter	<code>/x86_64 arm64 bpf/</code>
Capability Constraint	<code>/CAP_[A-Z_]+ SELinux context/</code>
Device Path	<code>/(dev proc sys)/[\w/]+</code>

Scalability in orchestrated environments is also a focus. Future deployments will integrate with Kubernetes via CRI hooks and namespace-aware policy enforcement, ensuring consistent, low-overhead protection across dynamic containerized clusters.

VII. CONCLUSION

This paper presented DeSFAM, a multi-phase, adaptive security framework for protecting containerized environments against syscall-level threats. Unlike static filtering approaches, DeSFAM combines static and dynamic syscall profiling, hybrid deep learning-based anomaly detection, contextual risk scoring, and real-time in-kernel enforcement using eBPF and LSM hooks.

At its core, DeSFAM integrates SyscallAD, a hybrid anomaly detection engine combining a Variational Autoencoder (VAE) and Isolation Forest to identify malicious syscall patterns based on frequency, sequence context, and timing features. The system also supports self-healing access list refinement, adapting syscall policies over time based on container-specific behaviors.

To enable threat-aware enforcement, DeSFAM maps syscalls to MITRE ATT&CK techniques and known CVEs, driving tiered enforcement strategies (log, rate-limit, block) based on dynamic risk scores. All policy enforcement is executed in-kernel, ensuring low-latency response and minimal performance overhead.

We evaluated DeSFAM on the DongTing dataset and real-world exploits (e.g., CVE-2021-4034 [25]). Results show high detection accuracy, sub-millisecond enforcement latency, and only 1.94% average CPU overhead. Compared to Confine, Optimus, and KubeRosy, DeSFAM achieves better syscall coverage, faster enforcement, and improved resilience. Our ablation study confirms that while Phase 1 enables foundational syscall reduction (62.4%), it is the combined effect of Phase 2’s real-time anomaly detection and Phase 3’s in-kernel enforcement that achieves complete CVE blocking (14/14) and zero false positives.

DeSFAM offers a scalable, workload-aware syscall defense model suitable for modern container infrastructures. Future enhancements include tighter Kubernetes integration, syscall argument validation, and automated response orchestration to further improve precision and adaptability.

APPENDIX A

RULE-BASED CVE PARSER FOR SYSCALL EXTRACTION

A. Scope and Design Principles

We design a deterministic, rule-based parser (rather than ML-based NLP) to extract high-risk syscalls from CVE descriptions and vendor advisories, ensuring:

- **Transparency:** Fully auditable and explainable pattern matching.
- **Precision:** Uses a kernel-specific syscall lexicon to avoid false positives.
- **Context-awareness:** Filters based on exploit conditions, kernel versions, architecture, capabilities, and device constraints.

The rule-based approach is preferred over ML-based NLP because it provides higher determinism, does not introduce false positives from model inference, and allows full inspection of each filtering decision. This is particularly critical in security contexts where auditability and explainability are paramount.

B. Syscall Dictionary Construction

The syscall dictionary is built directly from kernel sources:

Listing 6. Extracting syscall names from kernel sources.

```
# Generated from kernel v5.15 headers
grep -r 'SYSCALL_DEFINE' arch/x86/kernel/ \
| awk -F '(' '{print $2}' \
| cut -d, -f1 \
| sort -u
# --> [accept, access, acct, add_key, ...]
```

Sources:

- /arch/x86/entry/syscalls/syscall_64.tbl
- /include/linux/syscalls.h
- Linux man-pages Section II

This ensures that only valid, in-kernel syscalls are candidates for extraction and analysis. The resulting dictionary contains the canonical names of 335 syscalls supported by Linux kernel 6.x, together with common aliases (e.g., `getpid` and `sys_getpid`).

C. Extraction and Filtering Rules

Regex patterns are applied to CVE descriptions, vendor advisories, and security mailing list disclosures to identify syscall mentions and filter based on context.

Primary regex patterns:

Contextual filtering rules:

After extracting candidate syscalls, the parser filters them based on:

- 1) *Kernel version applicability:* Version ranges are extracted from patterns like “kernel versions prior to 5.12” and compared against the host kernel version.
- 2) *Architecture:* Mentions like “x86_64 only” or “ARM-specific” are matched and non-matching architectures are excluded.

Listing 7. Regex patterns for identifying syscall mentions in CVE descriptions.

```
# Pattern 1: Direct syscall mention (e.g., "via the ptrace() syscall")
SYSCALL_DIRECT = re.compile(
    r'\b(?:the\s+)?'
    r'(?P<syscall>' + '|'.join(SYSCALL_DICT) + r')'
    r'(?:(\d*))?\s*(?:system\s+call|syscall)',
    re.IGNORECASE
)

# Pattern 2: Function name in exploit context
SYSCALL_EXPLOIT = re.compile(
    r'(?:(exploit|abuse|invoke|call|use)\s+'
    r'(?:(the\s+)?'
    r'(?P<syscall>' + '|'.join(SYSCALL_DICT) + r')'
    r'(?:(\d*))?)',
    re.IGNORECASE
)

# Pattern 3: C annotation pattern (e.g., "setuid(0)" or "execve('/bin/sh', ...)")
SYSCALL_CODE = re.compile(
    r'\b(?:P<syscall>' + '|'.join(SYSCALL_DICT) + r')'
    r'\s*\([^\)]*\)',
    re.IGNORECASE
)
```

- 3) *Capability requirements:* CVEs requiring `CAP_SYS_ADMIN` or other elevated capabilities are excluded if the container lacks that capability.
- 4) *Device constraints:* Exploits requiring specific hardware devices (e.g., `/dev/kvm`, specific USB devices) are excluded if the device is unavailable.

D. Algorithm Pseudocode

E. Real-World Filtering Examples

Example 1: CVE-2022-0185 [31] (Linux Kernel — fsconfig Overflow)

- *CVE Description:* “A heap-based buffer overflow was found in the Linux kernel’s `legacy_parse_param` function in the Filesystem Context functionality...allows a local user with `CAP_SYS_ADMIN` to escalate privileges...”
- *Syscalls Extracted:* `ioctl`, `mount`
- *Filter Decision:* Requires `CAP_SYS_ADMIN` ⇒ retained in S_{blocked} if container has the capability; discarded otherwise.
- *Outcome:* For unprivileged containers, this CVE is filtered out. For containers with `CAP_SYS_ADMIN`, `mount` is added to S_{blocked} .

Example 2: CVE-2021-22555 [32] (NetFilter — heap overflow)

- *CVE Description:* “A heap out-of-bounds write affecting Linux since v2.6.19-rc1 was discovered in `net/netfilter/x_tables.c`...can be exploited with `setsockopt()` to achieve local privilege escalation.”
- *Syscall Extracted:* `setsockopt`
- *Version Condition:* Affects kernels < 5.12; discarded if host kernel is 5.15+.

Listing 8. Rule-based CVE parser: syscall extraction algorithm.
import re

```
def extract_high_risk_syscalls(cves, host_kernel,
    host_arch):
    """
    Extract the set of high-risk syscalls from CVE
    descriptions.

    Parameters:
        cves: List of CVE objects with a .desc field
        host_kernel: Host kernel version (tuple, e.g
            ., (6, 8, 0))
        host_arch: Host architecture (str, e.g., "
            x86_64")

    Returns:
        S_blocked: Set of syscall names to block
    """
    S_blocked = set()

    for cve in cves:
        # Filter 1: Check kernel version
        # applicability
        if not kernel_version_applicable(cve.desc,
            host_kernel):
            continue

        # Filter 2: Skip if architecture does not
        # match
        if architecture_mismatch(cve.desc, host_arch
            ):
            continue

        # Extract syscall mentions via regex
        found_syscalls = re.findall(syscall_regex,
            cve.desc)

        for syscall in found_syscalls:
            # Filter 3: Skip if special capability
            # required
            # but container does not have
            # it
            if requires_special_cap(cve.desc) and \
                not container_has_cap():
                continue

            # Filter 4: Skip if specific device
            # required
            # but not available
            if needs_specific_device(cve.desc) and \
                not device_available():
                continue

            # Syscall passed all filters -> add to
            # blocked set
            S_blocked.add(syscall)

    return S_blocked
```

- *Filter Decision:* If host kernel is 5.15, this CVE is skipped and `setsockopt` is *not* added to S_{blocked} , avoiding unnecessary blocking of legitimate network functionality.

These two examples illustrate how the context-aware CVE parser avoids both false positives (blocking safe syscalls in certain environments) and false negatives (missing dangerous syscalls when applicable). This deterministic approach ensures that S_{blocked} accurately reflects the true CVE attack surface of each deployed container

environment.

ACKNOWLEDGMENT

The authors acknowledge the anonymous reviewers for their valuable feedback and constructive suggestions, which significantly enhanced the clarity and quality of this manuscript. Special thanks are also extended to the FAST National University of Computer and Emerging Sciences, Karachi, Pakistan, for their technical guidance and infrastructure resources, which were instrumental in the implementation and evaluation of the DeSFAM framework.

CONFLICT OF INTEREST

The authors declare that there are no conflicts of interest related to the research, authorship, or publication of this manuscript. The research was conducted independently, and no external entity influenced the study design, data interpretation, or outcomes presented in this work.

REFERENCES

- [1] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, "A container security survey: Exploits, attacks, and defenses," *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [2] S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," in *IEEE Access*, vol. 7, 2019, pp. 52 976–52 996.
- [3] T. Combe, A. Martin, and R. Di Pietro, "To docker or not to docker: A security perspective," *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.
- [4] R. Shu, X. Gu, and W. Enck, "A study of security vulnerabilities on docker hub," in *Proceedings of the 7th ACM on Conference on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280.
- [5] NIST National Vulnerability Database, "CVE-2022-0847: Dirty pipe — linux kernel privilege escalation," <https://nvd.nist.gov/vuln/detail/CVE-2022-0847>, 2022.
- [6] —, "CVE-2019-5736: Docker runc container escape," <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.
- [7] B. Gregg, "BPF performance tools: Linux system and application observability," *Addison-Wesley Professional*, 2019, iISBN: 978-0-13-655482-1.
- [8] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [10] G. Duan, Y. Fu, M. Cai, H. Chen, and J. Sun, "DongTing: A large-scale dataset for anomaly detection of the Linux kernel," *Journal of Systems and Software*, vol. 203, p. 111768, Sep. 2023.
- [11] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1996, pp. 120–128.
- [12] K. Wang, J. J. Parekh, and S. J. Stolfo, "Anagram: A content anomaly detector resistant to mimicry attack," in *Proceedings of the 9th International Symposium on Recent Advances in Intrusion Detection (RAID)*, 2006, pp. 226–248.
- [13] G. Creech and J. Hu, "A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns," *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 807–819, 2014.
- [14] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.

- [15] W. Haider, J. Ahmad, M. Nasim, Z. A. Baig, and A. A. Laghari, "Methods for host-based intrusion detection with deep learning," *Digital Threats: Research and Practice*, vol. 2, no. 4, pp. 1–29, 2021.
- [16] M. El Khairi, D. Colle, F. De Turck, and B. Volckaert, "Contextualizing system calls in containers for anomaly-based intrusion detection," in *Proceedings of the ACM Cloud Computing Security Workshop (CCSW)*, 2022.
- [17] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, "DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time," *IEEE Access*, vol. 13, 2025.
- [18] S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.
- [19] C. Warrender, S. Forrest, and B. Pearlmutter, "Detecting intrusions using system calls: Alternative data models," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 1999, pp. 133–145.
- [20] D. Wagner and P. Soto, "Mimicry attacks on host-based intrusion detection systems," in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002, pp. 255–264.
- [21] A. S. Abed, C. Clancy, and D. S. Levy, "Applying bag of system calls for anomalous behavior detection of applications in Linux containers," in *Proceedings of the IEEE Globecom Workshops*, 2015.
- [22] P. Castanhel, P. Pinto, and F. Magalhães, "Taking a peek: An evaluation of anomaly detection using system calls for containers," in *Proceedings of the IEEE International Symposium on Computers and Communications (ISCC)*, 2021.
- [23] Q. Bsoul, M. Al-Quraan, and F. Al-Turjman, "Ensemble of random and isolation forests for graph-based intrusion detection in containers," in *Proceedings of the IEEE International Conference on Cyber Security and Resilience (CSR)*, 2022.
- [24] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, "Mining sequential patterns by pattern-growth: The PrefixSpan approach," *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 11, pp. 1424–1440, 2004.
- [25] NIST National Vulnerability Database, "CVE-2021-4034: Polkit local privilege escalation," <https://nvd.nist.gov/vuln/detail/CVE-2021-4034>, 2022.
- [26] M. Rostamipoor, S. Ghavamnia, and M. Polychronakis, "Confine: Fine-grained system call filtering for container attack surface reduction," *Computers & Security*, vol. 132, p. 103338, Sep. 2023.
- [27] S. Kim, B. J. Kim, and D. H. Lee, "Prof-gen: Practical study on system call whitelist generation for container attack surface reduction," in *Proceedings of the IEEE 14th International Conference on Cloud Computing (CLOUD)*, Sep. 2021, pp. 278–287.
- [28] S. Song, S. Suneja, M. V. Le, and B. Tak, "On the value of sequence-based system call filtering for container security," in *Proceedings of the IEEE 16th International Conference on Cloud Computing (CLOUD)*, Jul. 2023, pp. 296–307.
- [29] S. Yang, B. B. Kang, and J. Nam, "Optimus: Association-based dynamic system call filtering for container attack surface reduction," *Journal of Cloud Computing*, vol. 13, no. 1, p. 71, Mar. 2024.
- [30] J. Her, C. H. Jo, T. Park, and S. Lee, "KubeRosy: A dynamic system call filtering framework for containers," *IEEE Access*, vol. 12, pp. 159 889–159 901, 2024.
- [31] NIST National Vulnerability Database, "CVE-2022-0185: Linux kernel heap overflow in filesystem context," <https://nvd.nist.gov/vuln/detail/CVE-2022-0185>, 2022.
- [32] —, "CVE-2021-22555: Linux kernel netfilter heap out-of-bounds write," <https://nvd.nist.gov/vuln/detail/CVE-2021-22555>, 2021.